Mô hình xác suất Naive Bayes cho lexicon của rule engine và ảnh hưởng của nó lên hệ thống fusion

Báo cáo ngắn kỹ thuật (tiếng Việt)

SensitiveAI — Capstone Project
Báo cáo nội bộ cho mentor

21 tháng 10, 2026

Tóm tắt nội dung

Báo cáo này trình bày một hướng cải thiện hệ thống phát hiện ngôn từ thù địch tiếng Việt bằng một *mô hình xác suất* (Naive Bayes trên bag-of-words) sinh lexicon cho rule engine, thay cho việc gán nhãn thủ công hoặc bằng tần suất. Kết quả chính gồm hai phần, và phần thứ hai quan trọng hơn: (i) ở tầng **rule engine**, lexicon sinh bởi Naive Bayes nâng macro-F1 trên validation từ 0.4555 lên 0.4800 ($\Delta = +0.0245$, CI 95% $[+0.0012, +0.0476]$, $p = 0.038$) — **một cải thiện có ý nghĩa thống kê**; (ii) nhưng ở tầng **fusion** (hệ thống cuối), cùng thay đổi đó chỉ đưa macro-F1 test từ 0.6647 lên 0.6677 ($\Delta = +0.0030$, CI 95% $[-0.0035, +0.0094]$, $p = 0.351$) — **không có ý nghĩa**. Báo cáo phân tích vì sao: rule engine dù mạnh lên vẫn *yếu hơn* nhánh AI, và nút thắt thật nằm ở lớp OFFENSIVE. Chúng tôi trình bày kiến trúc hệ thống, công thức Naive Bayes, ngưỡng multi-label, toàn bộ chuỗi bốn hướng đã thử, cùng các hình minh hoạ. Đây là một *kết quả âm trung thực*, có giá trị định hướng: muốn nâng điểm hệ thống cuối, cần cải thiện nhánh AI/head cho lớp OFFENSIVE, không phải tiếp tục mở rộng lexicon.

1 Đặt vấn đề và câu hỏi nghiên cứu

Hệ thống của chúng tôi phân loại bình luận tiếng Việt thành ba lớp HATE / OFFENSIVE / CLEAN. Nhánh chính là một mô hình học sâu (encoder `microsoft/mdeberta-v3-base` + head nhiều góc nhìn). Song song, hệ thống có một **rule engine** dựa trên từ điển (lexicon): nó đối chiếu văn bản với danh sách term độc hại rồi cho điểm từng lớp. Hai nhánh được hợp nhất bằng một khối **fusion** (Hình **??**).

Lexicon hiện tại gồm **668 term** gán nhãn thủ công. Hạn chế hiển nhiên: nó nhỏ, tốn công, và mỗi term chỉ mang *một* nhãn. Mentor gợi ý:

> *"Có thể nằm ở nhiều label + dùng mô hình xác suất để tính xem từ đó thuộc nhãn nào?"*

Từ đó câu hỏi nghiên cứu của báo cáo:

Câu hỏi 1.
Một *mô hình xác suất* (Naive Bayes) có sinh được lexicon tốt hơn cách gán nhãn hiện tại, đo trên rule engine không?

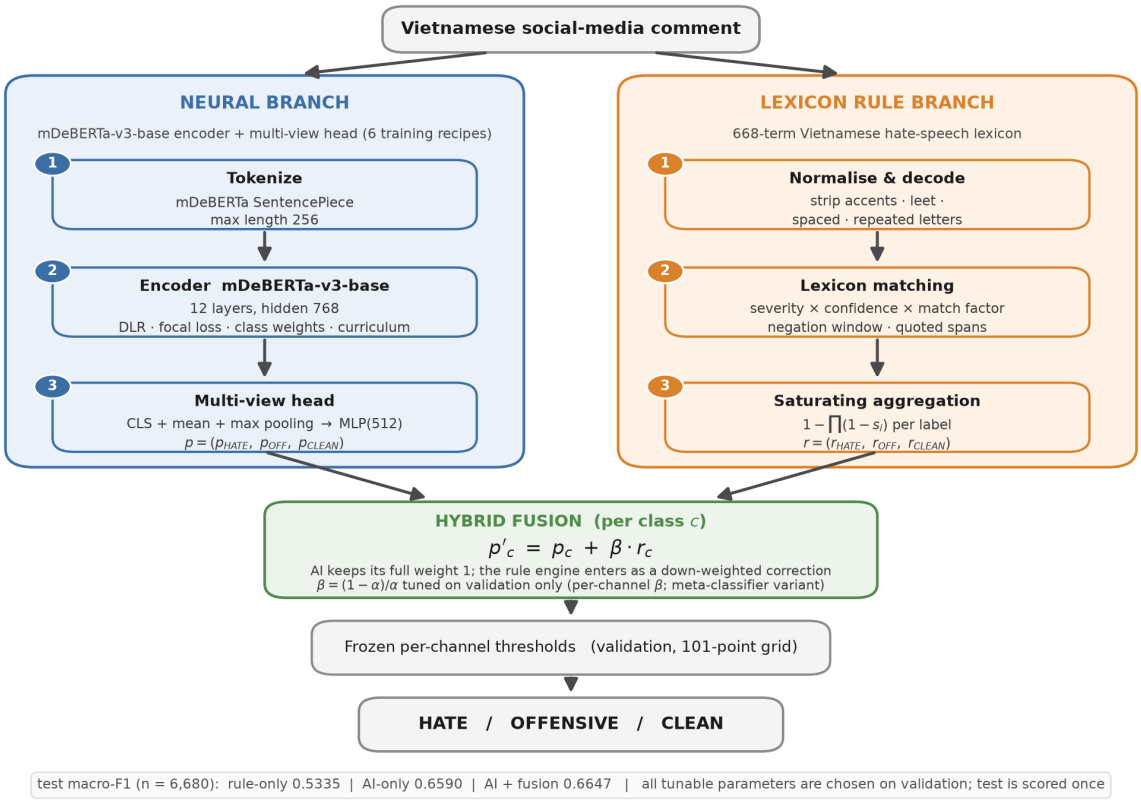
Câu hỏi 2.
Nếu rule engine tốt lên, *điểm hệ thống cuối* (fusion) có tốt lên tương xứng không?

Câu hỏi 3.
Multi-label có phải là nguồn cải thiện không, và nút thắt thật sự của cả hệ thống nằm ở đâu?

Trả lời ngắn gọn: (1) có, và có ý nghĩa thống kê; (2) **không** — cải thiện không truyền lên fusion; (3) multi-label *không* phải nguồn cải thiện, nút thắt thật là lớp OFFENSIVE và nhánh AI.

2 Kiến trúc hệ thống

Hình **??** vẽ kiến trúc thật của hệ thống. Có hai nhánh song song, được hợp nhất ở cuối.



Hình 1: Kiến trúc toàn hệ thống. **Nhánh AI** (trên): encoder `mdeberta-v3-base` + head nhiều góc nhìn (cls + mean + max) cho ra phân phối xác suất p. **Nhánh rule** (dưới): chuẩn hoá → lexicon → điểm rule r cho từng lớp, có xử lý obfuscation (mất dấu, leet, lặp ký tự). **Fusion** hợp nhất theo công thức $p' = p + \beta\, r$ (nhánh AI giữ trọng số 1, nhánh rule bị hạ trọng số β) rồi áp ngưỡng quyết định *theo từng kênh* (per-channel thresholds) đã chốt trên validation. Hình do `scripts/30_plot_system_architecture.py` sinh trực tiếp từ định nghĩa hệ thống (không vẽ tay).

2.1 Nhánh AI

Encoder `mdeberta-v3-base` (278,2M tham số) sinh biểu diễn chuỗi, sau đó head *nhiều góc nhìn* gộp ba vector: token đầu CLS, trung bình có mặt nạ (masked mean) và max có mặt nạ (masked max). Ba vector được nối thành một vector 2304 chiều, đi qua khối Linear$(2304 \to 512)$ → GELU → Dropout, rồi Linear$(512 \to 3)$ cho ra xác suất 3 lớp. Kết quả là phân phối $p = (p_{\text{HATE}}, p_{\text{OFFENSIVE}}, p_{\text{CLEAN}})$.

2.2 Nhánh rule engine

Nhánh này gồm ba bước:

1. **Chuẩn hoá văn bản**: Unicode NFC, viết thường, bỏ ký tự vô hình, gộp khoảng trắng, giới hạn ký tự lặp (nguuuu→nguu).
2. **Đối chiếu lexicon** qua nhiều "góc nhìn chống obfuscation": khớp chính xác, bỏ dấu, leet (`c0n ch0`), gộp ký tự lặp, chèn dấu phân cách, và fuzzy (khoảng cách ≤ 1). Mỗi kiểu khớp có một hệ số tin cậy riêng (`match_factors`).

3. **Cho điểm**: mỗi kết quả khớp có điểm

$$s = \underbrace{\text{severity}}_{\text{mức độ nhãn}} \times \underbrace{c}_{\text{confidence của term}} \times \underbrace{f}_{\text{hệ số kiểu khớp}} \times \underbrace{k}_{\text{hệ số ngữ cảnh (phủ định/trích dẫn)}},$$

rồi gộp về điểm từng lớp bằng tổng bão hoà

$$r_{\text{label}} = 1 - \prod_{i=1}^{n}(1 - s_i), \qquad n \le \text{aggregation_cap} = 10.$$

Công thức này *bão hoà*: càng nhiều term khớp thì điểm tiến gần 1 nhưng không vượt quá 1. Cuối cùng rule engine cho ra $r = (r_{\text{HATE}}, r_{\text{OFFENSIVE}}, r_{\text{CLEAN}})$.

2.3 Fusion

Fusion trộn hai nhánh. Bản đang dùng là *per-channel*:

$$p'_c = \alpha_c \, p_c + (1 - \alpha_c) \, r_c,$$

trong đó α_c là trọng số tin nhánh AI *riêng cho từng lớp* c, và r đã được "làm sạch" theo $r_{\text{clean}} = 1 - \max(r_{\text{hate}}, r_{\text{offensive}})$ để tránh mâu thuẫn giữa các lớp. Sau đó áp ngưỡng quyết định per-channel đã chốt trên validation. Trong báo cáo này, bước "quyết định" (ngưỡng) đã được tối ưu trên *validation* và *đóng băng* trước khi chấm *test*, đúng theo giao thức ở mục **??**.

3 Mô hình xác suất Naive Bayes cho lexicon

3.1 Ý tưởng

Thay vì gán nhãn lexicon bằng tay hoặc bằng tần suất thô, ta *học* phân phối xác suất "một term thuộc lớp nào" từ chính corpus huấn luyện (ViHSD + ViCM + Bontori, 135,238 dòng). Corpus được token hoá đơn giản thành *syllable* và *bigram/trigram*; mỗi term ứng viên (term cũ + term sinh bởi bước tokenize) được chấm điểm Naive Bayes.

3.2 Công thức

Với lớp $c \in \{\text{HATE}, \text{OFFENSIVE}, \text{CLEAN}\}$, gọi $\text{df}_c(\text{term})$ là số dòng thuộc lớp c chứa term, N_c là số dòng lớp c, V là kích thước từ vựng, α là hệ số làm trơn Laplace:

$$P(\text{term} \mid c) = \frac{\text{df}_c(\text{term}) + \alpha}{N_c + \alpha V}, \qquad P(\text{term}) = \frac{\text{df}(\text{term}) + \alpha}{N_{\text{all}} + \alpha V}.$$

Điểm quan trọng — dùng log-odds, không dùng posterior thô. Vì corpus rất lệch (khoảng 83% là CLEAN), nếu dùng posterior thô thì tiên nghiệm $P(c)$ sẽ kéo mọi term về CLEAN. Do đó ta dùng *lift*:

$$\ell_c(\text{term}) = \log P(\text{term} \mid c) - \log P(\text{term}).$$

Log-odds đã "chia" cho $P(\text{term})$ chung, nên nó đo mức term xuất hiện ở lớp c *nhiều hơn hẳn* các lớp khác bao nhiêu — đúng thứ ta cần cho lexicon. Sau đó chuẩn hoá về phân phối qua softmax:

$$P(c \mid \text{term}) = \frac{\exp\big(\ell_c - \max_{c'} \ell_{c'}\big)}{\sum_{c'} \exp\big(\ell_{c'} - \max_{c'} \ell_{c'}\big)}.$$

(Trừ max để ổn định số học, không đổi kết quả.)

3.3 Multi-label bằng ngưỡng

Cách cũ dùng arg max: mỗi term chỉ giữ *một* nhãn. Cách mới *multi-label*: giữ **mọi** lớp c có $P(c \mid \text{term}) \geq \theta$ (ngưỡng θ, ta gọi là `min_prob`). Đây chính là chỗ tận dụng hạ tầng có sẵn: matcher trong `src/rule_engine/matcher.py` đã lặp `for label in entry.labels`, nên một term hoàn toàn có thể mang hai nhãn HATE;OFFENSIVE và được cộng điểm cho cả hai lớp.

Ví dụ term "hai nhãn" thật trong dữ liệu: các tiếng chửi đa nghĩa như `dm`, `vl` xuất hiện nhiều ở *cả* hai lớp — đúng thứ mà arg max trước đây làm mất.

3.4 Lexicon thu được

Với $\alpha = 1.0$ và $\theta = 0.25$ (cấu hình khảo sát ban đầu), ta thu được **1,308 entry**, trong đó **585 term mang hai nhãn**. Phân bố top-1 trước khi multi-label hoá: HATE 804, OFFENSIVE 295, CLEAN 329. Một vài term xác suất cao (Bảng **??**).

Bảng 1: Ví dụ term có xác suất cao nhất theo mô hình Naive Bayes (script 40, corpus ViHSD+ViCM+Bontori). "Nhãn giữ" là các lớp có $P \geq \theta$ được đưa vào lexicon.

term	nhãn giữ	$P(\text{HATE})$	$P(\text{OFF})$	$P(\text{CLEAN})$
`thằng mặt`	HATE	0.907	0.062	0.030
`thằng chó`	HATE	0.902	0.082	0.016
`con mẹ mày`	HATE	0.898	0.069	0.033
`máy tui`	OFFENSIVE	0.040	0.882	0.078
`bủm chúa`	OFFENSIVE	0.063	0.876	0.061
`ăn cắp content`	OFFENSIVE	0.063	0.876	0.061

4 Thiết lập thực nghiệm và giao thức chống rò rỉ

Toàn bộ thí nghiệm tuân thủ quy tắc: **chỉ tune trên validation; test chỉ chấm một lần và không ghi đè số cũ**.

- **Dữ liệu**: 2,672 dòng validation, 6,680 dòng test.
- **Tune**: hệ số làm trơn α và ngưỡng multi-label θ được quét trên *validation* (script 42). Ngưỡng quyết định của rule engine cũng tái tối ưu trên validation.
- **Đóng băng**: sau khi chọn cấu hình tốt nhất trên validation, ngưỡng được *đóng băng* rồi mới chấm test (script 17 chế độ test).
- **Không ghi đè**: mọi artifact mới ghi vào thư mục riêng (`rule_engine_nb/`, `fusion_nb/`); số liệu SOTA cũ giữ nguyên.
- **Kiểm định**: paired bootstrap (macro-F1) + McNemar, 10,000 bootstrap, dùng `src/evaluation/analysis.py`.

Hình **??** (Hướng 4) và Bảng **??** cho thấy lexicon NB là hướng duy nhất vượt kiểm định. Hình **??** là forest plot của toàn bộ chuỗi.

5 Kết quả

5.1 Tầng 1 — Rule engine: NB tốt lên CÓ Ý NGHĨA

Bảng **??** so sánh lexicon hiện tại với lexicon NB, đo trên rule engine *đứng riêng* ở validation.

Bảng 2: Lexicon sinh bởi Naive Bayes vs lexicon hiện tại, trên rule engine đứng riêng (validation, $n = 2672$). "Argmax" là biến thể mỗi term chỉ một nhãn; cột chính "Macro-F1 (thr)" tái tối ưu ngưỡng trên validation.

Lexicon	Entry	Macro-F1 (thr)	Macro-F1 (argmax)	Acc	F1 HATE	F1 OFF	F1 CLEAN
current (668)	668	**0.4555**	0.3438	0.702	0.368	0.169	0.830
nb only (1308)	1308	**0.4782**	0.3881	0.755	0.404	0.168	0.863
current + nb	1310	0.4596	0.3432	0.754	0.362	0.153	0.864
nb multilabel only	585	0.4498	0.4191	0.808	0.327	0.122	0.900

Diễn giải:

- **nb only** (dùng *toàn bộ* lexicon NB): 0.4782 vs 0.4555 $\Rightarrow \Delta = +0.0227$, CI 95% $[+0.0013, +0.0438]$, $p = 0.037$ (McNemar $234/92$, $p < 0.001$) $\Rightarrow$ **CÓ ý nghĩa**.
- **current + nb** (giữ cả lexicon cũ): $\Delta = +0.0042$, **CI chứa 0,** $p = 0.702$ $\Rightarrow$ không ý nghĩa. Nghĩa là lexicon NB *thay thế* tốt hơn là *bổ sung* vào lexicon cũ.
- **nb multilabel only** (chỉ giữ 585 term mang *hai* nhãn): $\Delta = -0.0057$, **CI chứa 0** $\Rightarrow$ giữ riêng term hai nhãn là *không* đủ — chốt lại multi-label *không* phải nguồn cải thiện.

5.2 Tầng 1 (tuned) — điểm tốt nhất trên validation

Script 42 quét $\alpha \times \theta$ trên validation (Hình **??**, cột "Hướng 4 tuned"). Kết quả tốt nhất: $\alpha = 0.3$, $\theta = 0.35$ $\Rightarrow$ **1,155 entry, 208 term hai nhãn**, macro-F1 **0.4800** ($\Delta = +0.0245$, CI 95% $[+0.0012, +0.0476]$, $p = 0.038$; McNemar $235/78$).

Một quan sát quan trọng: **9/12** cấu hình đều cải thiện rule engine, và macro-F1 gần như phẳng theo cặp (α, θ) — trừ cấu hình $\theta = 0.50$ **(0 term hai nhãn) sụp** -0.049. Điều này cho thấy: *mô hình xác suất* mới là thứ giúp, còn multi-label chỉ là *hiệu ứng phụ nhỏ*; và cần θ đủ thấp để giữ được các term hai nhãn.

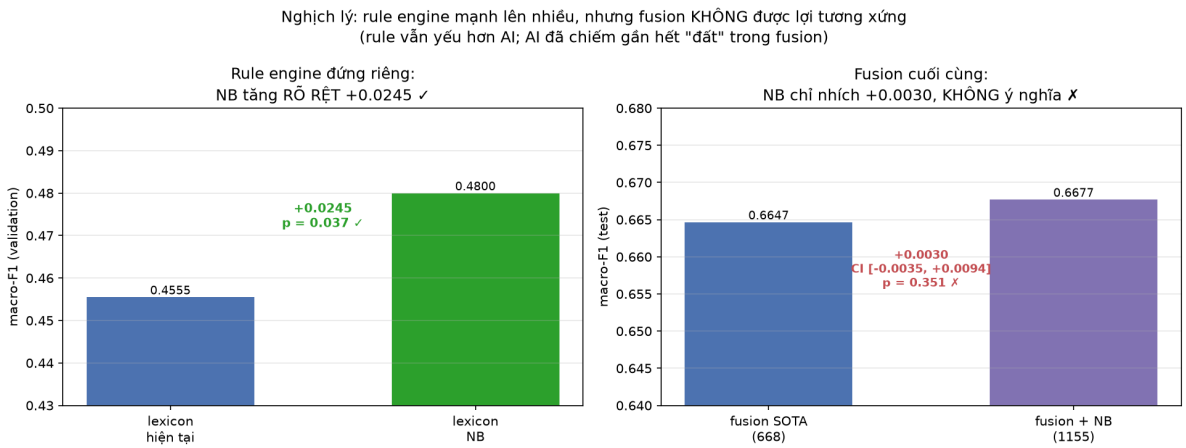
5.3 Tầng 2 — Fusion: cải thiện KHÔNG truyền lên

Ta cắm lexicon NB (đã tune, 1155 entry) vào đúng nhánh rule của hệ thống, giữ nguyên nhánh AI (dùng lại cache dự đoán, *không* train lại), rồi chấm fusion trên test. Bảng **??**.

Bảng 3: Đo trên **test** ($n = 6680$), cả hai hệ thống dùng chung cache dự đoán AI, nên chênh lệch *chỉ* đến từ lexicon của rule engine.

Hệ thống	Macro-F1	F1 HATE	F1 OFF	F1 CLEAN
fusion SOTA (lexicon 668)	0.6647	0.5920	0.4612	0.9408
fusion + NB (lexicon 1155)	**0.6677**	0.6084	0.4535	0.9412
Δ	**+0.0030**	+0.0164	−0.0077	+0.0004

Kiểm định: $\Delta = +0.0030$, CI 95% $[-0.0035, +0.0094]$, $p = 0.351$; McNemar 29 đúng /25 sai ($p = 0.684$) $\Rightarrow$ **KHÔNG có ý nghĩa**.



Hình 2: Nghịch lý cốt lõi. *Trái*: trên rule engine đứng riêng, lexicon NB tăng **+0.0245** ($p = 0.037$, có ý nghĩa). *Phải*: trên fusion cuối cùng (test), cùng thay đổi chỉ nhích **+0.0030** (không ý nghĩa).

6 Phân tích: vì sao cải thiện "bốc hơi" ở fusion?

Đây là phần quan trọng nhất. Rule engine mạnh lên $+0.047$ trên test ($0.5335 \rightarrow 0.5805$) nhưng fusion chỉ nhích $+0.0030$. Có ba lý do, đều đo được:

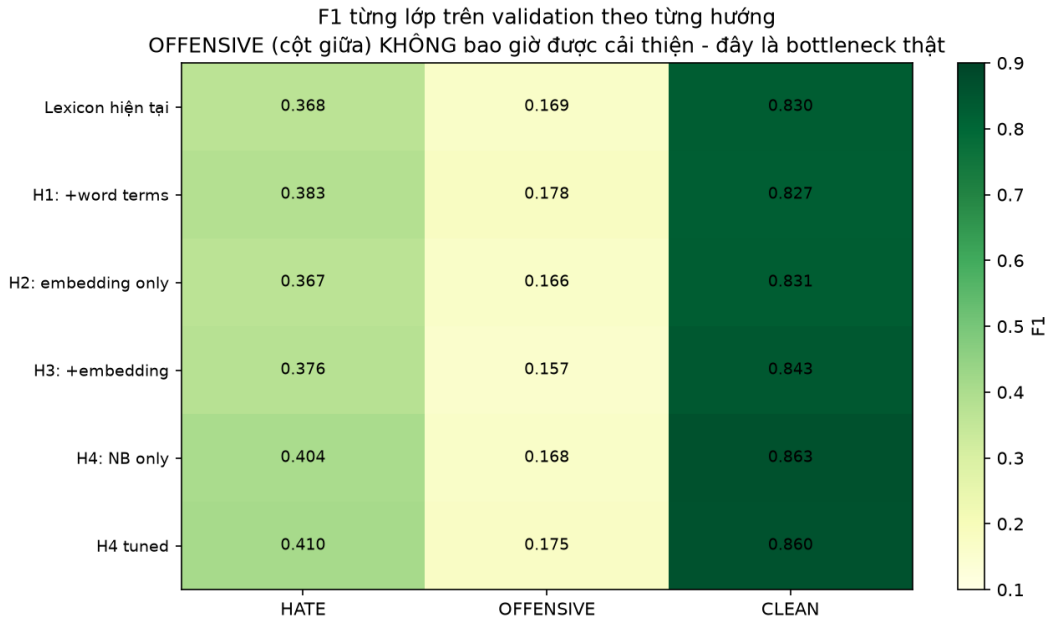
(1) Rule engine vẫn yếu hơn nhánh AI. Trên test, rule engine NB đạt macro-F1 0.5805 còn nhánh AI đạt 0.6590 (và biến thể AI tốt nhất 0.6638). Rule dù mạnh lên vẫn là "thành phần yếu". Khi trộn một thành phần yếu (dù tốt hơn trước) vào một thành phần mạnh, cải thiện biên thu được rất nhỏ.

Bằng chứng định lượng: fusion *đã nhận ra* rule mạnh lên và *chuyển trọng số* sang rule — trọng số tổng α đổi từ 0.9 (SOTA) xuống 0.7 (NB), và ở kênh HATE trọng số AI tụt xuống $\alpha_{\text{HATE}} = 0.1$ (gần như tin rule hoàn toàn). Nhờ vậy F1 HATE trên test tăng $0.5920 \rightarrow 0.6084$. Nhưng fusion *không thể* đẩy macro-F1 lên thêm.

(2) Rule engine vẫn rất yếu ở lớp OFFENSIVE. F1 OFFENSIVE của rule NB trên test chỉ 0.3142. Khi fusion chuyển bớt trọng số sang rule, F1 OFFENSIVE của hệ thống *giảm* ($0.4612 \rightarrow 0.4535$). Đây là cái giá của việc tin rule hơn.

(3) Nhánh AI đã chiếm gần hết "đất". Khoảng trống giữa AI-only (0.6590) và fusion (0.6647) chỉ 0.006. Rule mạnh lên chỉ lấp được khoảng $1/2$ khoảng trống đó, và phần lấp vào lại bị lớp OFFENSIVE ăn mất.

Nút thắt thật. Hình **??** cho thấy F1 lớp OFFENSIVE *không* được cải thiện bởi *bất kỳ* hướng lexicon nào ($0.169 \rightarrow 0.175$). Đây là nút thắt thật của cả hệ thống — và nó nằm ở nhánh AI/head, không ở rule engine.



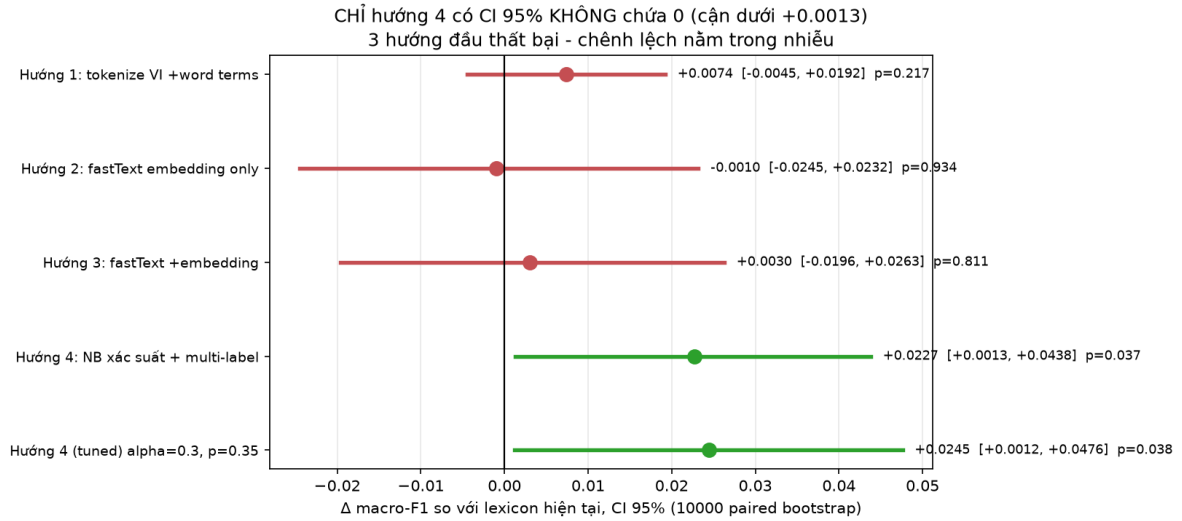
Hình 3: Heatmap F1 từng lớp trên validation theo từng hướng lexicon. Cột giữa (OFFENSIVE) *không bao giờ* đổi màu — mọi hướng đều bó tay ở lớp này. Đây là bằng chứng trực tiếp cho thấy nút thắt nằm ở OFFENSIVE.

7 Toàn bộ chuỗi bốn hướng đã thử

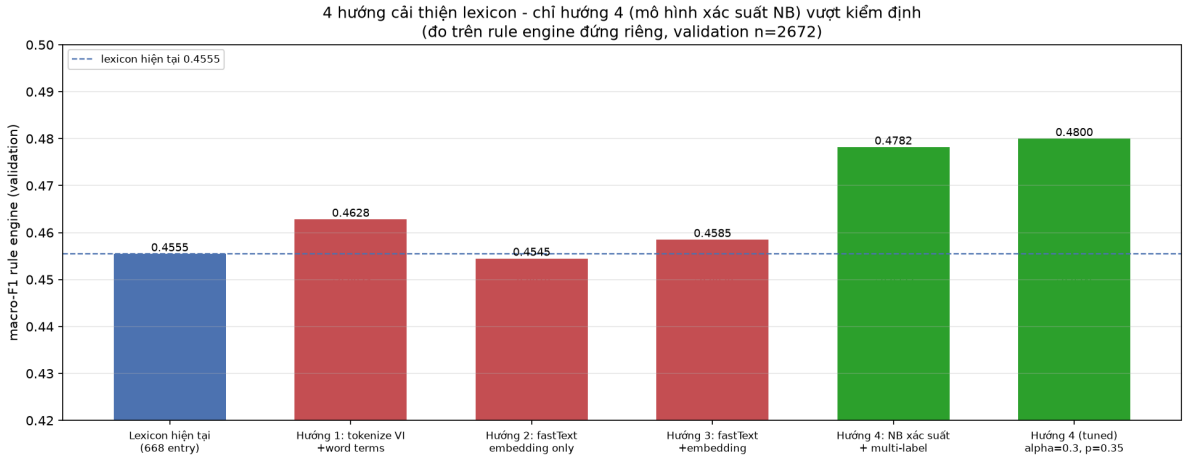
Trước khi đi tới Naive Bayes, chúng tôi đã thử ba hướng khác (đều theo gợi ý mở rộng lexicon). Bảng **??** và Hình **??** tổng hợp tất cả, đo trên rule engine đứng riêng ở validation.

Bảng 4: Bốn hướng cải thiện lexicon, đo trên rule engine đứng riêng (validation, $n = 2672$). 10,000 paired bootstrap. Chỉ Hướng 4 vượt kiểm định. (Dòng cuối là số đo trên fusion ở test.)

Hướng	Macro-F1	Δ	CI 95%	Kết luận
Lexicon hiện tại (mốc)	0.4555	—	—	—
H1: tokenize VI + word terms	0.4628	+0.0074	$[-0.0045, +0.0192]$	**không**
H2: fastText embedding only	0.4545	-0.0010	$[-0.0245, +0.0232]$	**không**
H3: fastText + embedding	0.4585	+0.0030	$[-0.0196, +0.0263]$	**không**
H4: NB xác suất + multi-label	0.4782	+0.0227	$[+0.0013, +0.0438]$	**có**
H4 tuned ($\alpha = 0.3, \theta = 0.35$)	**0.4800**	+0.0245	$[+0.0012, +0.0476]$	**có**
Fusion SOTA (test)	0.6647	—	—	—
Fusion + NB (test)	0.6677	+0.0030	$[-0.0035, +0.0094]$	**không**



Hình 4: Forest plot: Δ macro-F1 và CI 95% của *mọi* hướng so với lexicon hiện tại. **Chỉ Hướng 4 có CI 95% không chứa 0** (cận dưới $+0.0013$); ba hướng còn lại CI đều chứa 0, tức chênh lệch nằm trong nhiễu.



Hình 5: macro-F1 của rule engine qua từng hướng (validation). Cột xanh lá = hướng vượt kiểm định (Hướng 4), cột đỏ = hướng thất bại. Đường đứt là mốc lexicon hiện tại 0.4555.

Vì sao ba hướng đầu thất bại? Cả ba đều mắc một trong hai lỗi thiết kế: H1 (tokenize) vẫn dựa trên *tần suất tài liệu*, và term mới học được (138) không bù nổi term bị mất (270); H2/H3 (fastText) dùng *cosine* rồi ép về một nhãn bằng arg max — cosine đo "hai vector có gần nhau không" nên mọi lớp đều rơi vào khoảng 0.44–0.49, không tách được. Naive Bayes sửa đúng chỗ đó: nó đo "term này xuất hiện ở lớp này *nhiều hơn hẳn* lớp khác không" (log-odds), nên tách được term có tín hiệu thật.

8 Kết luận và bài học

1. **Câu hỏi 1 — Có:** mô hình xác suất Naive Bayes sinh lexicon tốt hơn cách gán nhãn hiện tại. Trên rule engine đứng riêng (validation), $0.4555 \rightarrow 0.4800$, $\Delta = +0.0245$, $p = 0.038$. Đây là hướng *duy nhất* trong bốn hướng vượt kiểm định.

2. **Câu hỏi 2 — Không:** cải thiện **không truyền lên fusion**. Trên test, fusion chỉ nhích $0.6647 \rightarrow 0.6677$ ($\Delta = +0.0030$, $p = 0.351$).

3. **Câu hỏi 3 — Multi-label không phải nguồn cải thiện.** Cấu hình tốt nhất có 208 term hai nhãn, nhưng khi ép $\theta = 0.50$ (bỏ hết term hai nhãn) điểm *sụp* -0.049; ngược lại "chỉ giữ term hai nhãn" cũng kém. Thứ giúp là *mô hình xác suất*, không phải multi-label. Nút thắt thật là lớp OFFENSIVE.

Bài học trung tâm. *Cải thiện một thành phần không tự động truyền lên hệ thống cuối.* Rule engine tăng $+0.047$ (test) nhưng fusion chỉ tăng $+0.0030$ (không ý nghĩa), vì rule vẫn yếu hơn AI và AI đã chiếm gần hết "đất" trong fusion.

Hàm ý cho hướng đi tiếp theo. Để nâng điểm hệ thống cuối, cần cải thiện **nhánh AI/head cho lớp OFFENSIVE**, chứ không phải tiếp tục mở rộng lexicon. Cụ thể: (i) dữ liệu / hàm mất mát tập trung vào ranh giới HATE–OFFENSIVE; (ii) hiệu chỉnh xác suất (calibration) cho lớp thiểu số; (iii) tái thiết kế decoder dùng cấu trúc (hai tầng: toxic/not $\rightarrow$ hate/offensive).

A Artifact và script tái lập

Mọi số trong báo cáo đều đọc trực tiếp từ các tệp sau (không tính lại bằng tay):

Nội dung	Tệp / lệnh
Sinh lexicon NB (script 40)	`outputs/lexicon_experiments/nb_multilabel/nb_report.json`
Đo rule engine (script 41)	`.../nb_multilabel/nb_eval.json`
Tune α, θ (script 42)	`.../nb_multilabel/nb_tuning.json`
Kiểm định fusion (script 43)	`.../nb_multilabel/fusion_nb_significance.json`
Hình tổng kết (script 44)	`outputs/lexicon_experiments/figures/directions_*.png`
Bảng tổng hợp	`outputs/lexicon_experiments/directions_summary.md`
Rule engine NB (validation/test)	`outputs/evaluation/rule_engine_nb/`
Fusion NB (test)	`outputs/evaluation/fusion_nb/`
Config	`configs/rule_engine_nb.yaml`

Trình tự tái lập (chạy trong thư mục gốc dự án):

```
python scripts/40_nb_multilabel_lexicon.py
python scripts/41_evaluate_nb_multilabel.py
python scripts/42_tune_nb_lexicon.py
python scripts/17_evaluate_rule_engine.py --config configs/rule_engine_nb.yaml \
    --split validation --out outputs/evaluation/rule_engine_nb
python scripts/17_evaluate_rule_engine.py --config configs/rule_engine_nb.yaml \
    --split test --out outputs/evaluation/rule_engine_nb
python scripts/19_plot_rule_engine.py --config configs/rule_engine_nb.yaml \
    --out outputs/evaluation/rule_engine_nb --split validation
python scripts/19_plot_rule_engine.py --config configs/rule_engine_nb.yaml \
    --out outputs/evaluation/rule_engine_nb --split test
python scripts/23_evaluate_fusion.py --rule-eval-dir outputs/evaluation/rule_engine_nb \
    --out-dir outputs/evaluation/fusion_nb --reports-dir outputs/reports_fusion_nb \
    --model sensitiveai-vi-custom-curriculum
python scripts/43_significance_fusion_nb.py
python scripts/44_plot_improvement_directions.py
```